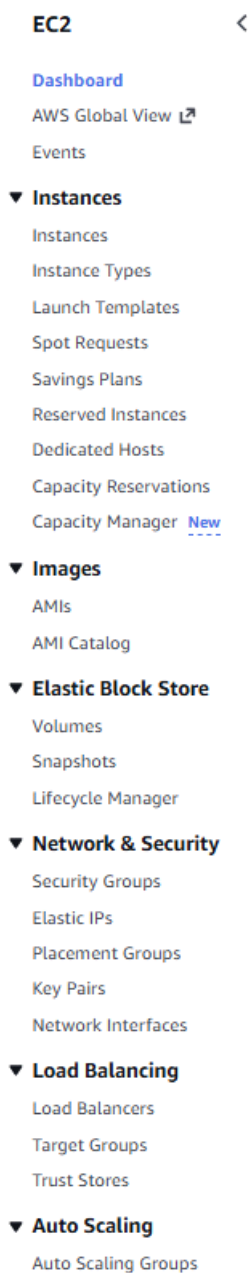


EC2 (Region scoped)

20 November 2025 06:17

All below points are important for EC2 -



Instances (create => connect => manage) -

- While launching the instance we need to provide below information -

Category	Details / Fields	Notes
General Instance Info	Name	User-defined identifier for instance
	AMI (Amazon linux, Ubuntu etc.)	Defines OS, pre-installed apps, configuration
	Instance Type (t2.micro, t3.micro, etc.)	Defines CPU, RAM, network performance
	Key Pair	Private key used for SSH connection
Network Configuration	VPC	Logical network environment
	Subnet	Sub-network (AZ-specific) within VPC
	Security Group	Virtual firewall controlling inbound/outbound rules
	Allow Traffic	Common rules: SSH (22), HTTP (80), HTTPS (443)
Storage	Root Volume	Default EBS volume attached to instance
	Additional Volumes	Optional EBS volumes for extra storage
Lifecycle	Create → Connect → Manage	Full process flow while dealing with instances

- EC2 lifecycle -
pending → running → stopping → stopped → terminating → terminated

- When we launch an EC2 instance in the usual way, it is considered an **On-Demand Instance**. This means we create the instance based on our immediate requirement ("on demand"). On-Demand instances offer flexibility but are **more expensive** compared to other purchase options.

For **background or non-critical workloads**, we can use **Spot Instances** instead.

Spot Instances work on a **capacity-based pricing model**, where AWS sells unused EC2 capacity at a **significantly lower price** (up to 90% cheaper). You request a Spot instance, and if AWS has available spare capacity, your instance launches. If capacity reduces later, AWS may **terminate the instance with a 2-minute warning**.

Spot Instances are ideal when:

- Workloads are fault-tolerant
 - Workloads can be interrupted
 - You want the lowest possible compute cost
- To create a spot instance we need to create **spot-fleet-request** using below steps -
 - Go to EC2 Console → Spot Requests → Create Spot Fleet Request.
 - Choose allocation strategy (capacityOptimized recommended).
 - Set target capacity (Spot + optional On-Demand).
 - Select multiple instance types for flexibility.
 - Configure launch settings – AMI, instance type, VPC/subnets, SGs, key pair, storage.
 - Choose IAM Fleet Role (AWSServiceRoleForEC2SpotFleet).
 - (Optional) Set max spot price or leave blank.
 - Choose request type – maintain (auto-replace) or request (one-time).
 - Add tags (cost/owner/env).
 - Review and submit the Spot Fleet request.

Category	Description
Purpose	Launch multiple Spot/On-Demand instances across instance types to meet target capacity.
Pricing	Uses Spot pricing (up to 90% cheaper). Can mix On-Demand + Spot.
Strategy Options	lowestPrice, capacityOptimized (recommended), diversified, priceCapacityOptimized.
Capacity Types	Target capacity, On-Demand capacity, Spot capacity, instance weighting.
Instance Flexibility	Allows multiple instance types → higher chances of fulfillment.
Launch Spec Includes	AMI, Instance Type, VPC/Subnet, SGs, Key Pair, Storage, User Data.
IAM Role Required	AWSServiceRoleForEC2SpotFleet.
Request Types	maintain (auto-replace terminated spot instances) or request (one-time).
Use Cases	Fault-tolerant workloads, analytics, CI jobs, large scale compute.
Risk	Spot instances can be interrupted with a 2-minute warning.
Best Practices	Provide 3–6 instance types, use multiple subnets/AZs, use capacityOptimized strategy.

• Instance Types

What: Combinations of CPU, memory, storage and networking (e.g., t3, m6i, c6g, r5) tuned for different workloads.

Families

- General purpose: t3, m5, m6i
- Compute optimized: c5, c6g
- Memory optimized: r5, x1
- Storage optimized: i3 (NVMe)
- Accelerated: p3, g4 (GPU), inf1 (inference)

• Reserved instances -

Reserved Instances (RIs) are a **billing discount** that you purchase for EC2, RDS, ElastiCache, Redshift, and OpenSearch. They **do NOT reserve capacity** (except Zonal RIs).

They **reduce the hourly cost** of instances when you commit for **1-year or 3-year** term.



Why Reserved Instances?

Because On-Demand instances are expensive for long-running workloads.

With RIs you can save:

- Up to 75% cheaper** than On-Demand pricing
- Savings depend on:
 - Term (1-year / 3-year)
 - Payment option (No Upfront / Partial / All Upfront)
 - Standard vs Convertible RI



How RIs Work Internally?

RIs apply **automatically** to running instances that match:

- Instance family (e.g., t3)
- Size (t3.medium, t3.large etc.)
- OS (Linux/Windows)
- Tenancy (Default/Dedicated)
- Region or AZ (depending on RI type)

They reduce the bill—nothing to configure on EC2.

🔗 Types of Reserved Instances

1 Standard RIs

- ✓ Max cost savings (up to 75%)
- ✓ Best for predictable workloads
- ✓ Can modify **AZ, instance size, network type**
- ✗ Cannot change instance family (t3 → t4g)

Use case:

Stable prod workloads running 24/7.

2 Convertible RIs

- ✓ Can change instance *family* (t3 → m5 → c6g)
- ✓ More flexible
- ✓ Up to 54% savings
- ✗ Lower discount compared to Standard RI

Use case:

When instance type may change over time.

🌐 Zonal RI vs Regional RI

Zonal : Tied to **specific Availability Zone**

Regional : Applies at **region level**

💰 Payment Options

Payment Option	Discount	Description
All Upfront	Highest	Pay 100% at start
Partial Upfront	Medium	Pay some now, rest monthly
No Upfront	Lowest	Pay monthly, commit only

• Savings plans -

AWS Savings Plans are a **flexible pricing model** that gives up to **72% cost savings** in exchange for committing to a **consistent hourly spend** (e.g., \$5/hour) for **1 or 3 years**.

Unlike Reserved Instances, Savings Plans are **not tied to a specific instance type** and offer far more flexibility.

You commit to spend a fixed amount (like ₹50/hour).

AWS then **automatically applies discounts** to your compute usage up to that amount.

✓ When to Use Savings Plans?

Use when:

- Workloads run **24/7**
- Architecture may change (migrate to Fargate/Lambda)
- You're unsure about long-term instance types
- You want flexibility + cost optimization

🔍 Savings Plans vs RIs (Crisp Comparison)

Feature	Savings Plans	Reserved Instances
Flexibility	★ High	Low–Medium
Applies To	EC2, Lambda, Fargate	Mainly EC2/RDS/etc
Savings	Up to ~66%	Up to ~75%
Capacity Reservation	✗ No	✓ Yes (Zonal RI)
Instance Binding	✗ No	✓ Yes (Standard RI)

• When to use spot vs on-demand vs reserved vs savings plans -

Model	Use When	Savings	Tolerance
On-Demand	Unpredictable, short-term workloads	Least	No interruptions
Spot	Fault-tolerant, flexible workloads	Highest	Can be interrupted
Reserved Instances	Predictable 24/7 workloads	Max (up to 75%)	No interruption, fixed setup
Savings Plans	Need flexibility in instance types/services	High (~66%)	No interruptions

Launch template

EC2 **Launch Template** is a reusable, versioned blueprint that defines **how your EC2 instances should be launched**.

It contains all the configuration parameters you normally provide when launching an instance manually.

🎯 Why Launch Templates?

Launch Templates solve **3 major problems** in EC2 management:

1. Consistency

You don't want each developer launching EC2 instances with random:

- AMIs
- Security Groups
- Key Pairs
- IAM roles
- Instance types

Launch templates enforce **standardization**.

2. Version Control

You can have **multiple versions** of the same template:

Example:

- v1 → t2.micro, AMI-1 (old app)
- v2 → t3.medium, AMI-2 (new app)
- v3 → add user-data, change SG, etc.

You can **roll back** anytime → This is important for autoscaling and fleets.

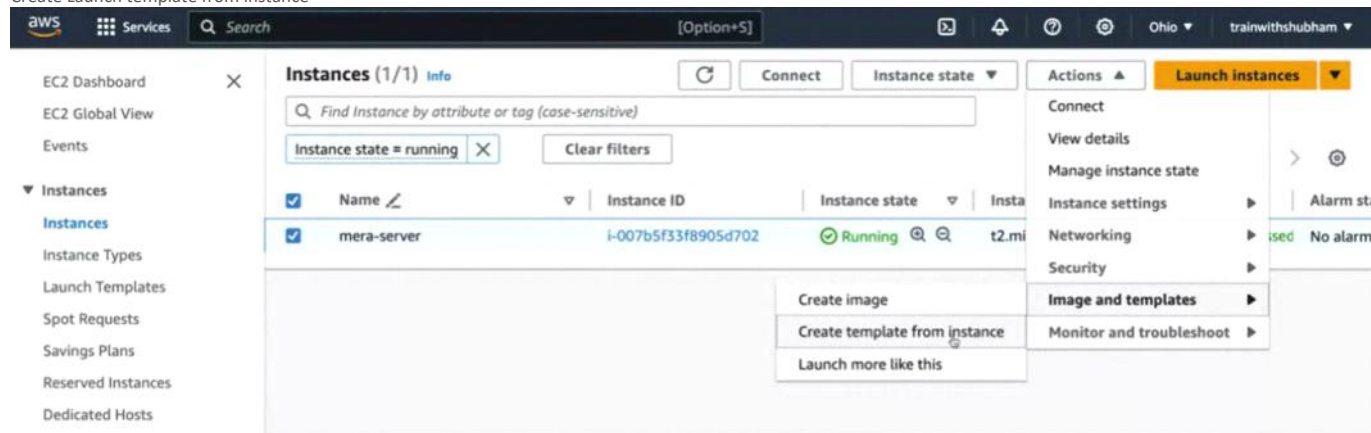
3. Automation

Launch templates are required by:

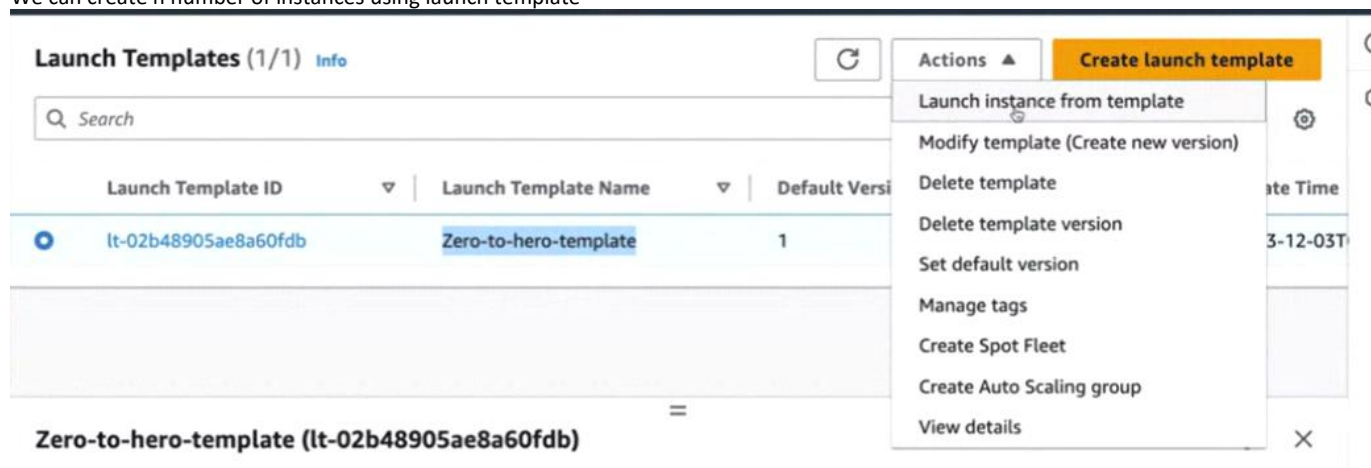
- **Auto Scaling Groups (ASG)**
- **EC2 Spot Fleets**
- **EC2 Fleet**
- **AWS Batch**
- **EKS Worker Nodes**
- **EMR**

Launch Configurations (older) are deprecated because **Launch Templates support all new EC2 features**.

Create Launch template from instance -



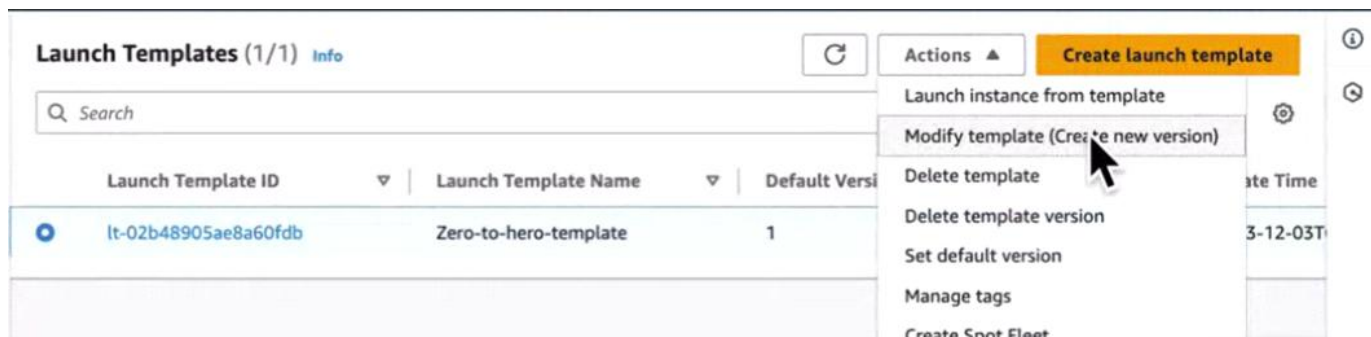
We can create n number of instances using launch template -



Launch template deals with only those things we did while creating the instance. There is no track for launch template, what we did after creating an instance. If we want to make some process like install nginx as part of launching instance itself then we need to use **"user data scripts"**.

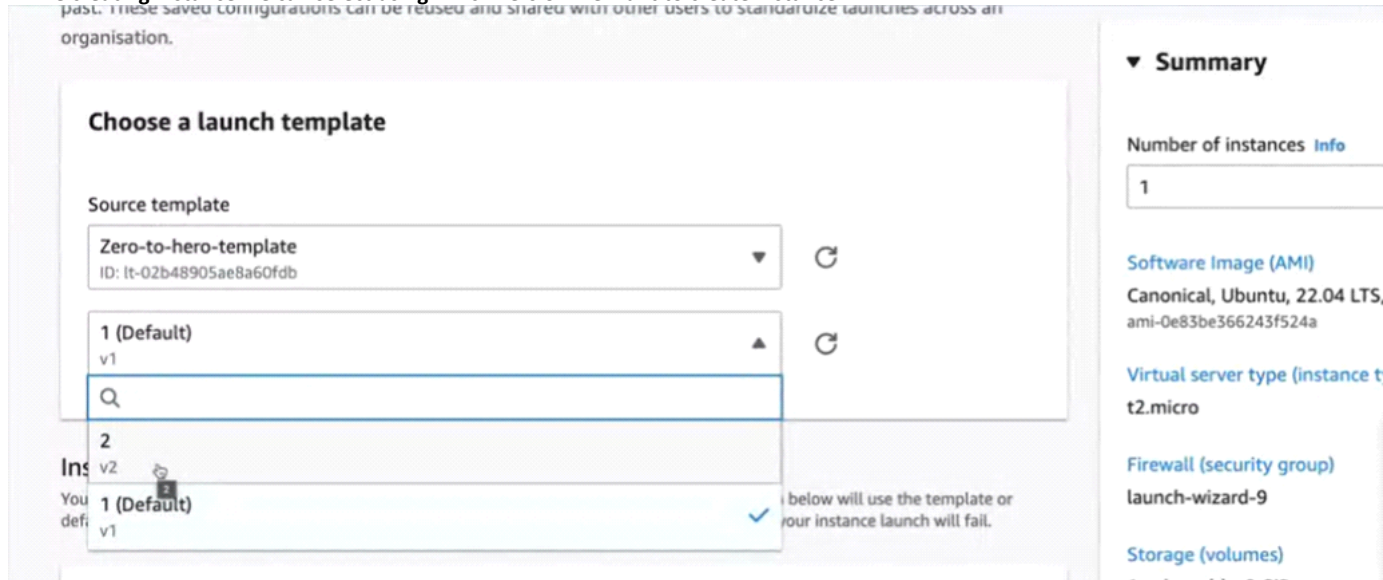
Instead of **adding user data while launching instance using launch template**, we can make it a part of launch template.

Modify the launch template -



Change the version from let's say v1 to v2.

While creating instance we can select using which version we want to create instance -



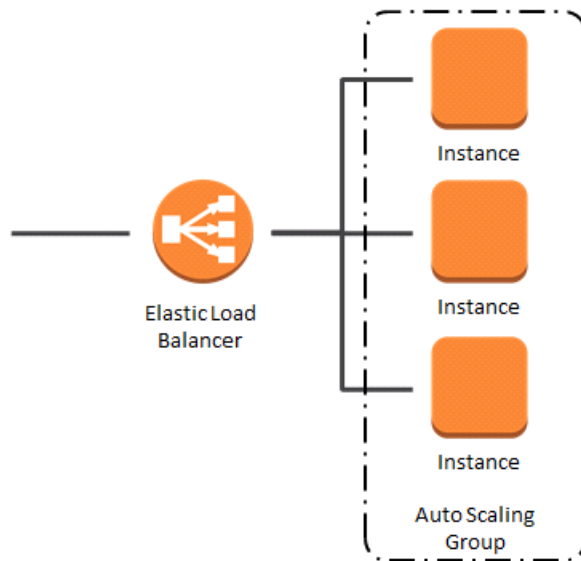
Go to advanced details and then add user data script (shell script).

Some important points about user-data script -

- **User Data** is a script that runs **automatically at the first boot** of an EC2 instance. User-data runs only **once** during the **first launch** of an instance.
- It supports shell-scripts and powershell-scripts
- Must Start with a Shebang (`#!/bin/bash`)
- Runs as root user
- How to Re-run User Data on Every Boot?

```
#!/bin/bash
echo "true" > /var/lib/cloud/instance/sem/config_scripts_user
```
- User-data is processed by **cloud-init** package in Linux.

Auto scaling group and Load balancing -



🚀 End-to-End Flow: EC2 → Launch Template → ASG → Target Group → Load Balancer

This is the standard architecture for **highly available, auto-scaling, fault-tolerant** applications on AWS. Let's break it step-by-step in the actual direction of flow and responsibilities.

🏠 1. Launch Template — *Blueprint for EC2*

A **Launch Template (LT)** defines *how* your EC2 instances should be created.

It contains:

- AMI (app image)
- Instance type
- Security groups
- IAM instance profile
- User-data script
- Storage mappings
- Network settings
- Tags
- Purchasing option (Spot/On-demand)

Purpose

The LT is like the “blueprint” used by the ASG to create EC2 instances **consistently** and **predictably**.

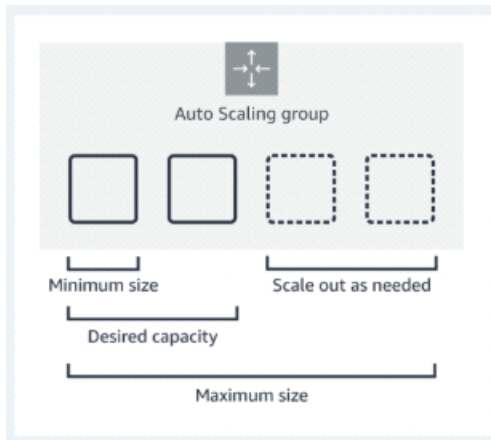
ASG never creates EC2 directly — it always uses a Launch Template.

🏠 2. Auto Scaling Group (ASG) — *Creates & Manages EC2 Instances*

The ASG uses the **Launch Template** and maintains a required number of EC2 instances (Desired Capacity).

An **ASG defines**:

- Minimum instances (high availability)
- Maximum instances (cost control)
- Desired instances (normal load)
- Scaling policies (CPU, ALB request count, SQS depth, etc.)
- Health checks (ELB + EC2)
- Subnets (multi-AZ distribution)
- Target Groups attachment



ASG responsibilities

- Launch EC2 instances using LT
 - Replace unhealthy instances
 - Distribute instances across AZs
 - Scale out/in based on metrics
 - Register/deregister instances in Target Group automatically
- ASG = Auto-healing + Auto-scaling layer**

3. Target Group — Where EC2 Instances Are Registered

A **Target Group** is a logical grouping of backend targets:

- EC2 instances
- IPs
- Lambda
- (For ALB/NLB)

For ASG:

When ASG launches EC2:

1. Instance is **added to Target Group**
2. Registered target undergoes **health checks**:
 - HTTP/HTTPS for ALB
 - TCP/UDP for NLB
3. Only healthy instances receive traffic

Why Target Group exists?

Because:

- Load Balancer should not directly query ASG
 - Instances come and go dynamically
 - LB needs real-time health & availability info
- The Target Group acts as the “bridge” between ASG and the Load Balancer.

4. Load Balancer (ALB/NLB) — Entry Point for All Traffic

The **Load Balancer** distributes traffic to the Target Group.

Application Load Balancer (Layer 7)

- HTTP/HTTPS
- Routing based on path, host, headers
- Web apps, microservices

Network Load Balancer (Layer 4)

- TCP/UDP
- High throughput / low latency
- Gaming/VoIP/real-time services

LB responsibilities:

- Accept traffic from public users
 - Apply listener rules (:80, :443)
 - Forward to the **Target Group**
 - Only send traffic to **healthy** targets
- User → Load Balancer → Target Group → EC2 instance

🚀 AMI vs User Data vs Launch Template

An **Amazon Machine Image (AMI)** is a **template** used to launch EC2 instances. It contains everything required for the instance to boot:

Think of AMI as a **"golden image"** for launching pre-configured servers repeatedly.
You can create an AMI from:

- Running EC2 instance ("Create Image")
- Stopped instance
- EBS Snapshot

Create from Running Instance

1. Install required software
2. Harden security
3. Clean logs (optional)
4. Stop background services not required at boot
5. Go to EC2 → Actions → Create Image
6. AWS takes a snapshot of the root volume

! Important

Creating AMI from **running** instance = **crash-consistent snapshot**

Creating AMI from **stopped** instance = **file-system-consistent snapshot** (better)

Feature	AMI	User Data	Launch Template
Purpose	Preconfigured machine image to launch EC2	Boot-time script to configure instance	Blueprint defining full instance configuration
Contains	OS, patches, software, configs, baked-in dependencies	Shell commands / cloud-init script executed at first boot	AMI ID, instance type, networking, volumes, security groups, KMS, tags
Executed?	Not executed — it's just an image	Yes, executed ONCE at first boot	Not executed — used as a parameter set
Examples	Custom web server AMI, hardened OS AMI, golden image	Install packages, pull code, run setup commands	Used by ASG, Spot Fleet, EC2 to launch consistent servers
Speed	Fastest boot — everything baked	Slow (must download + install at boot)	Medium — depends on selected AMI + user-data
Mutability	Immutable (image is fixed)	Mutable (script can fail/change)	Immutable template with versions
Best for	Production workloads, scaling, golden images	Dynamic configurations, per-instance customizations	Autoscaling, DR, launching many identical EC2s
Versioning	No versioning (manually managed)	No versioning	Versioning built-in
Managed By	EC2 / EBS snapshot	cloud-init	EC2 console, ASG, CLI, IaC
Portability	Region-specific	OS-specific (Linux/Windows)	Region-specific but portable via API

Amazon EBS (Elastic Block Store) — In-Depth EC2 Revision Notes

(Volumes + Snapshots)

Amazon EBS is a **high-performance, durable block storage service** that attaches to EC2 instances.

It behaves like a **virtual SSD/HDD** for your instance.

An EC2 instance can have:

- **One root volume** (must support boot)
- **Multiple additional EBS data volumes**

Volumes exist within **same AZ can be attached to EC2 instance**.

Can attach/detach to running instances

You can change:

- Volume size
- Volume type
- IOPS

No downtime (for most OS).

EBS Snapshots

An EBS Snapshot is a **point-in-time backup** of an EBS volume stored in **Amazon S3 (managed by AWS)**.

Key Properties:

- **Incremental**
Only changed blocks are saved → efficient storage.
- **Durable**
Stored in S3 with 11 9s durability.
- **Region-level service**
Snapshots exist at **region** level, not AZ.

How Snapshots Internally Work

First Snapshot:

- Full copy of the entire EBS volume.

Subsequent Snapshots:

- Only changed blocks stored (incremental).

Restore:

- AWS reconstructs using all snapshot deltas.

Example:

1. Take snapshot of Production DB volume
2. Create new volume from snapshot
3. Attach to staging EC2
4. Run DB testing using prod-like data

- By-default EBS root volume will also get deleted when server is terminated.

Setting:

- **DeleteOnTermination = true** (default)
- **false** → keeps root volume after termination

Used when:

- You want logs/debug data even after server terminates.

- EBS volume cannot be attached to two EC2 instances at the same time.
- AWS does support multi-attach, but only for io1/io2 volumes. With Multi-Attach, a volume can attach to up to 16 EC2 instances in the same AZ.

- EBS volumes can be modified **online** without stopping the instance

- Volume expansion happens in **3 layers**:

- a. EBS volume
- b. Partition
- c. Filesystem

- **Shrink is not supported**, only expansion

- **growpart** expands the partition

- **resize2fs / xfs_growfs** expands the filesystem

- Always ensure that:

- OS supports resizing
- Volume is gp2/gp3/io1/io2

EC2 Elastic IP (EIP) -

An **Elastic IP (EIP)** is a **static, public IPv4 address** provided by AWS that you can allocate to your AWS account and assign to an EC2 instance or network interface.

Unlike a normal public IP (which changes after stop/start), **Elastic IP never changes** unless you release it.

Why AWS Created Elastic IPs?

Because when you stop & start an EC2 instance:

- The instance gets a **new public IP**
- DNS mappings break
- Applications break
- Developers using IP directly face downtime
- Load balancer health checks fail

Elastic IP solves this → **persistent public IP for your instance.**

★ Key Characteristics

1. Static Public IPv4
2. Attached to an EC2 Instance or ENI (Elastic Network Interface)
Best practice → Assign EIP to ENI, attach ENI to any EC2.
3. Automatically remappable

If instance fails:

- Move EIP from unhealthy instance → healthy instance
- Takes < **1 minute**
- Zero DNS changes required

Elastic IP is free only when:

- ✓ It is **attached to a running EC2 instance** AND
- ✓ Only **1 EIP per instance**

You are charged when:

- ✗ EIP is allocated but **not attached to any instance**
- ✗ Instance is **stopped**
- ✗ You attach **multiple EIPs** to the same instance

AWS wants to reduce exhaustion of IPv4 pool, so they penalize unused public IPs.

📁 Lifecycle of an Elastic IP

- 1 Allocate an EIP → belongs to AWS account
 - 2 Associate it → attach to instance / ENI
 - 3 Use it → instance gets a fixed public IP
 - 4 Disassociate → returns to your account pool
 - 5 Release → gives it back to AWS
- (Once released, you may not get it again)

Note: Elastic IPs do not exist for IPv6.

📁 In Summary

Elastic IP is a static, public IPv4 that you can allocate, associate, and remap across EC2 instances.

It remains the same even when the instance stops/starts.

AWS charges for unused EIPs to prevent hoarding of IPv4 addresses.

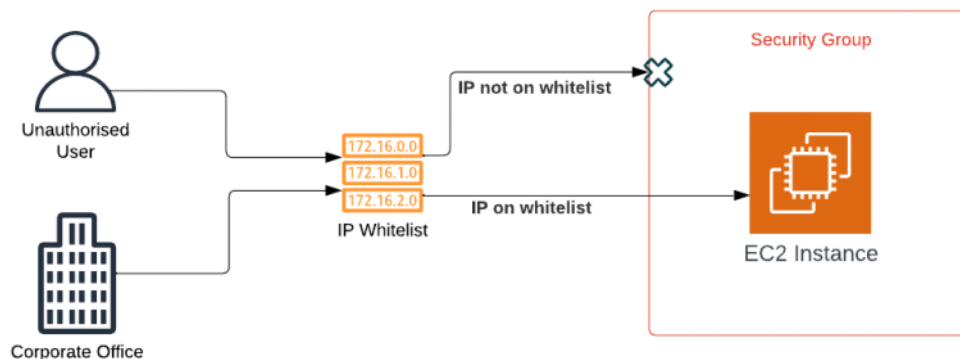
Best used for bastion hosts, VPN endpoints, NAT, and failover architectures.

Not recommended for scalable production apps — use load balancers instead.

AWS Security Groups

AWS **Security Groups (SGs)** are **stateful, virtual firewalls** that control **inbound and outbound** traffic *to and from* EC2 instances, ENIs, RDS, Lambda (via ENI), ECS tasks, etc.

They operate at the **instance/network interface level**, not at subnet level (that's NACL). SG is associated with VPC. If we create SG under VPC-1 then it is not available under VPC-2.



AWS Security Group with IP whitelisting

🔑 Key Characteristics

1. Instance-Level Firewall
2. SECURITY GROUPS ARE STATEFUL

This is *the most important point* in interviews.

Stateful → Return traffic is automatically allowed.

Example:

- You allow **inbound port 80**
- The outbound response is automatically allowed
- No need to explicitly allow outbound 80

This is opposite of NACLs (which are stateless).

3. Deny Rules Do NOT Exist

Security groups support:

- **Allow rules only**
- Implicit **deny everything else**

This prevents misconfigurations like conflicting rules.

4. Security Groups Support Only “Permit” Logic

You **cannot block** an IP or port using SGs.

To block:

- Use **NACLs**
- Use **WAF**
- Use **Firewall Manager**
- Use **Security appliance (Palo Alto)**

5. Operate at Instance Network Interface (ENI)

Each ENI can have:

- One primary SG
- Additional SGs (up to 5 by default)

Rules are **OR merged**:

If SG-1 and SG-2 both apply → combined allow rules apply.

Inbound vs Outbound Rules

Inbound = Who can talk to my instance?

Example:

- Allow SSH (22) from your office IP
- Allow HTTP (80) from 0.0.0.0/0
- Allow MySQL (3306) only from app server's SG

Outbound = Where can my instance go out to?

Default outbound is **allow all (0.0.0.0/0)** so instance can:

- Download packages
- Connect to DB
- Hit external API

Summary for Interview

Security Groups are stateful, instance-level virtual firewalls in AWS that only allow "Allow" rules. They control inbound and outbound traffic, evaluate rules using OR logic, automatically allow return traffic, and are used to secure EC2, RDS, ENIs, ALBs, and ECS tasks. They enforce least privilege access using IP ranges or other SG references.

VPC (Region scoped)

24 November 2025 06:26

☒ Understand IP address ranges and CIDR

IPv4 is made of 32 bits.

Example:

192.168.255.255

In bits form:

11000000 10101000 11111111 11111111

- Minimum value -> 00000000 i.e. 0
- Maximum value -> 11111111 i.e. 255

Hence max value of ip range is 255.

☒ Example

Suppose I want to get 100 servers for my company then I can choose the IP range:

From **192.168.255.0 to 192.168.255.255** => **total 255 servers**.

If we observe closely then first 24 bits in the above range are static (constant) and that's why the above range can be defined in CIDR as: **192.168.255.0/24**

In case we want to keep first 16 bits static (we want more servers) then CIDR will be : 192.168.255.0/16

Create VPC -

Let's say we have created VPC with CIDR - **192.168.0.0/16 (192.168.0.0 to 192.168.255.255)**

Now we can create sub-networks out of it to assign to different departments.

Sub-Network 1 -

192.168.1.0 to 192.168.1.255

i.e. 192.168.1.0/24

Sub-Network 2 -

192.168.2.0 to 192.168.2.255

i.e. 192.168.2.0/24

Sub-Network 3 -

192.168.3.0 to 192.168.3.255

i.e. 192.168.3.0/24

Sub-Network 4 -

192.168.4.0 to 192.168.4.255

i.e. 192.168.4.0/24

While creating subnet -

- We need to select VPC under which we are creating subnet.
- We need to select AZ (**VPC is region scoped but subnet is AZ scoped**)

Now let's create an EC2 instance under this VPC -

VPC and subnet selection -

▼ Network settings [Info](#)

VPC - required [Info](#)

vpc-0d4bdf8949260d50d (youtube-vpc)
192.168.0.0/16

[Create new VPC](#)

Subnet [Info](#)

subnet-0a96db841dcd508da NETWORK - 1
VPC: vpc-0d4bdf8949260d50d Owner: 392833926319
Availability Zone: us-east-1a IP addresses available: 251 CIDR: 192.168.1.0/24

[Create new subnet](#)

subnet-0a96db841dcd508da NETWORK - 1 ✓

subnet-08444af0e5c5786eb NETWORK - 2
VPC: vpc-0d4bdf8949260d50d Owner: 392833926319
Availability Zone: us-east-1b IP addresses available: 251 CIDR: 192.168.2.0/24

subnet-041f030ba3fb53dc9 NETWORK - 3
VPC: vpc-0d4bdf8949260d50d Owner: 392833926319
Availability Zone: us-east-1a IP addresses available: 251 CIDR: 192.168.3.0/24

subnet-06287e1ee6381f4a2 NETWORK - 4
VPC: vpc-0d4bdf8949260d50d Owner: 392833926319
Availability Zone: us-east-1b IP addresses available: 251 CIDR: 192.168.4.0/24

Description - required [Info](#)

We need to create a new SG as we know SG is associated with VPC and there is no SG created under this new VPC.

After creating the instance if we try to ssh (connect to an instance) it will fail. As we have created instance inside VPC

and VPC does not have internet access. Instances inside the same VPC can communicate with each other but not with the outside world.

To give the internet access to our VPC we need to create Internet gateway (IGW). Go to IGW and just by providing

some name we can create IGW. It will show in detached state. We need to attach VPC to our IGW -

Internet gateways (1/2) [Info](#)

[Filter internet gateways](#)

	Name	Internet gateway ID	State	VPC ID
☒	youtube-internet-g...	igw-0f0feab210a6a34ed	Detached	-
☐	-	igw-0f420a9c1f76031b7	Attached	vpc-014612dcf708df290

[Actions](#) [Create internet gateway](#)

[View details](#)

[Attach to VPC](#)

[Detach from VPC](#)

[Manage tags](#) 19

[Delete internet gateway](#) 19

After attaching the IGW now we need to change the routes in route table -

By default any traffic cannot go outside VPC. We need to allow traffic from our subnet range to go outside and access internet.

Hence we need to edit the route for the route table attached to our VPC. We need to attach IGW to the route.

VPC > Route tables > rtb-0cc7926fe00bd5dca > Edit routes

Edit routes

Destination	Target	Status	Propagated
192.168.0.0/16	local	Active	No
0.0.0.0/0	igw-0f0feab210a6a34ed	-	No

[Add route](#)

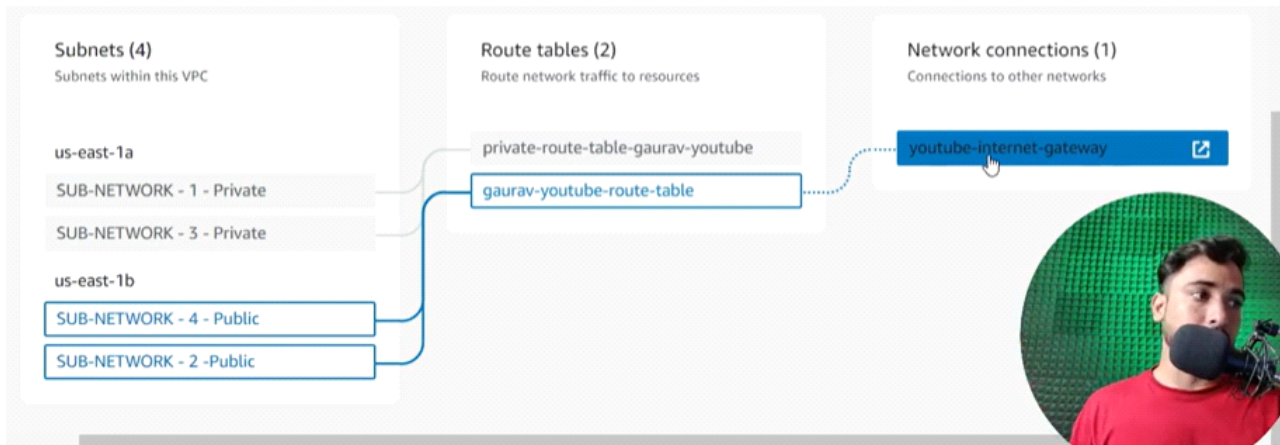
[Cancel](#) [Preview](#) [Save changes](#)

Now we can ssh to the instance.

Public subnet - Subnet with internet access.
Private subnet - Subnet without internet access.

How to create a private subnet?

Create a route table under our VPC with name let's say private-route-table. Keep the default route only (no internet access). Now go to subnet-1 and subnet-3 (defined in above section), and associate private-route-table to it. Now instances created inside subnet-1 and subnet-3 will not have internet access. Keep servers with your sensitive data in private subnet like database server etc.



Jump server (bastion host) -

A bastion host is a server whose purpose is to provide access to a private network from an external network, such as the Internet. Because of its exposure to potential attack, a bastion host must minimize the chances of penetration.

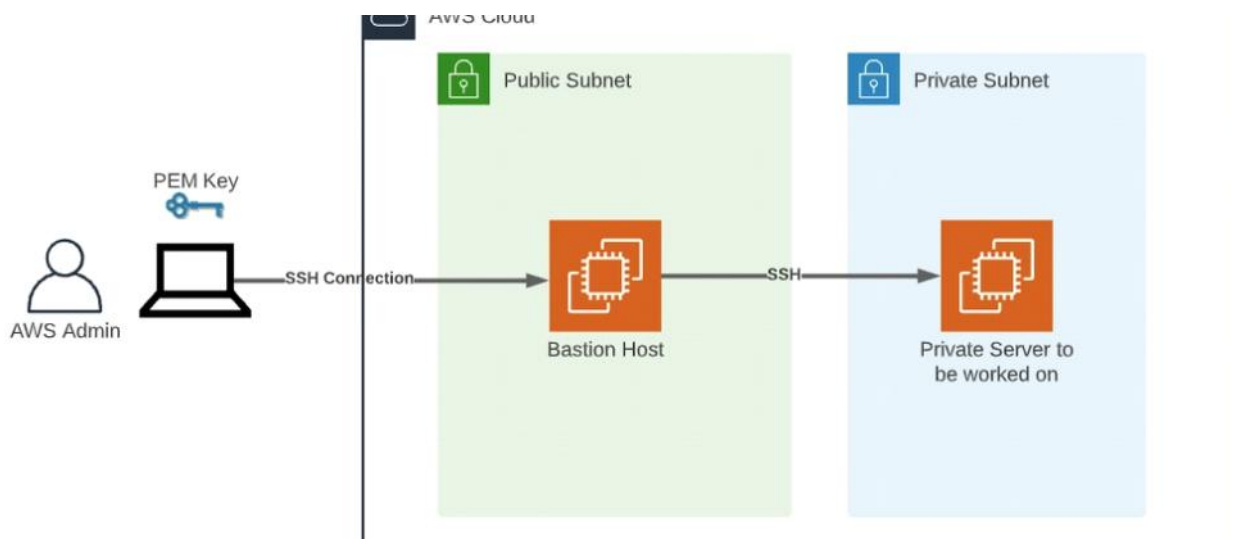
Why to use ?

If we want to SSH into EC2 instance on the private subnet from our home/office (or using a development machine), currently we can't. Our instance has no public IP, it is in a Private Subnet (no direct route from the internet). This is where we can use a Bastion Server or Jump server.

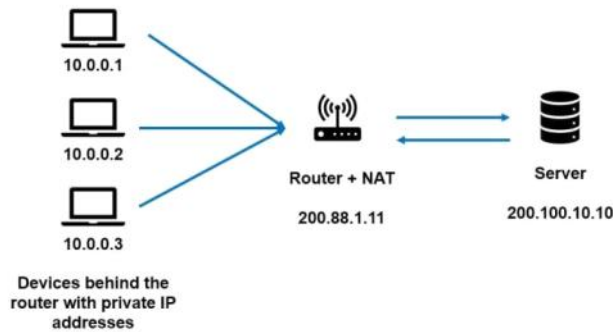
How to use?

We will create an instance in public subnet (jump server) and ssh to the same. Now we will copy private pem key attached to our private server and run ssh command using the key. We will now be able to connect to our private server from jump server.

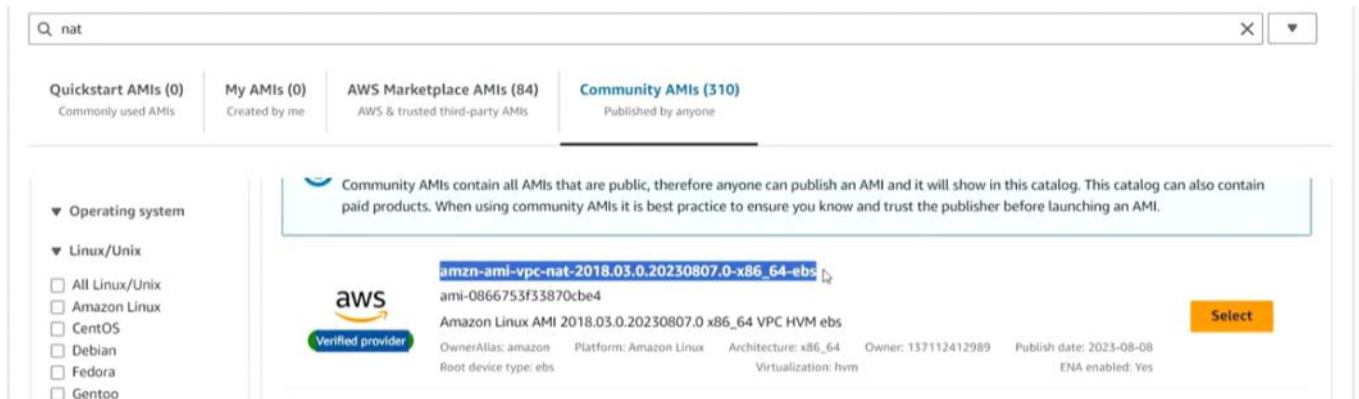
But after connecting to private server if we try to curl google.com then we will not get any response because private instance does not have internet access.



NAT server (not recommended) -

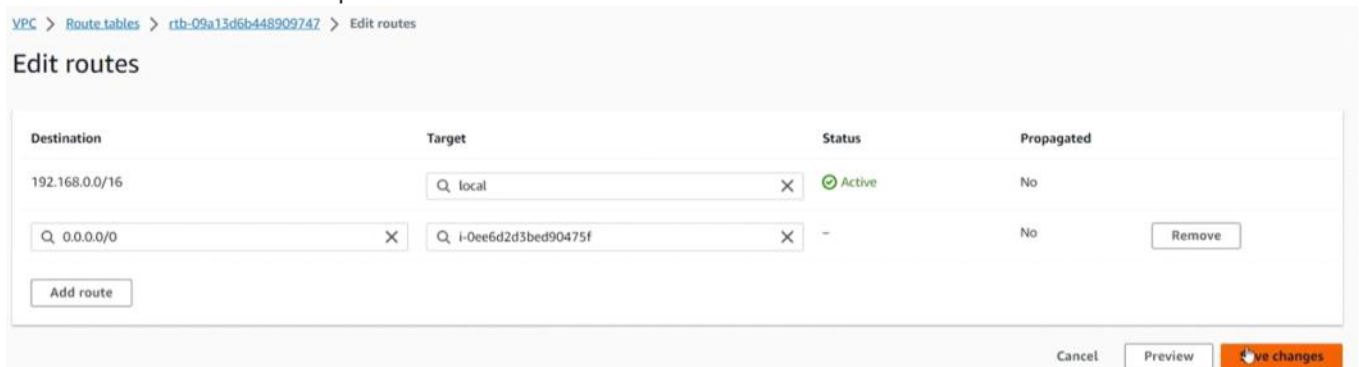


Launch a new ec2 instance with verified aws-vpc-nat ami -

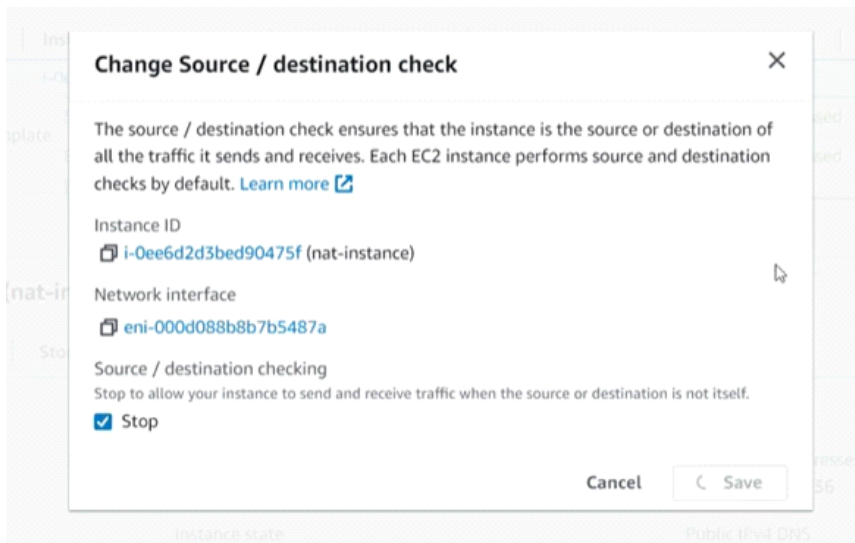


Select our vpc and subnet should be public of course !
Now we have NAT instance created.

Now we need to route internet access via NAT instance for the private servers. For that we need to add one route in route table attached to our private subnet.



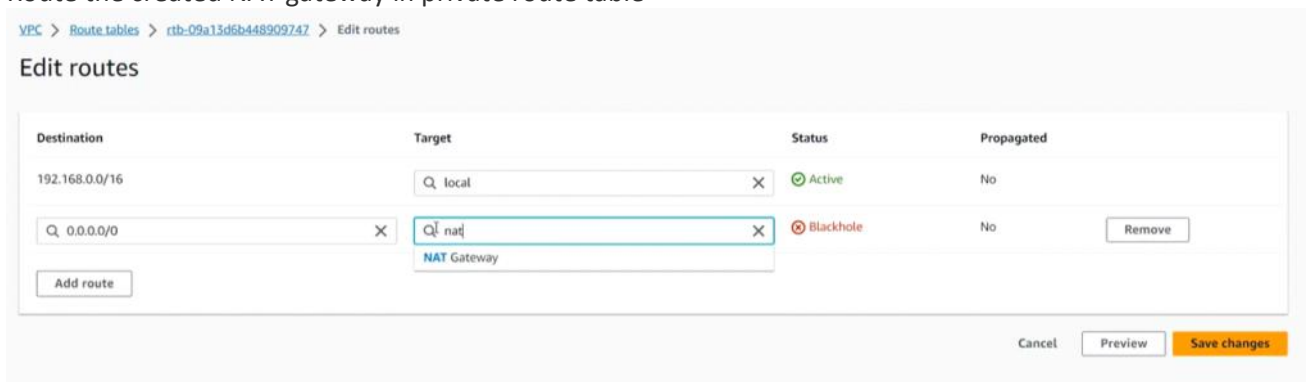
Now if we try to access internet from private server it will still not work. Because source (NAT instance) will try to access google.com but the response is expected to be delivered at destination (private server) hence NAT will not accept the traffic which is not destined to it. To accept such traffic we need to stop source-destination check for NAT instance -
Select instance --> network settings --> change source/destination check



Now internet will be accessible in private server. But this is manual way to achieve internet access in private server which needs maintenance. AWS recommends use of **NAT gateway** instead.

NAT Gateway - How to create?

Go to NAT gateways --> provide some name --> select VPC --> select public subnet --> allocate elastic IP. Route the created NAT gateway in private route table -

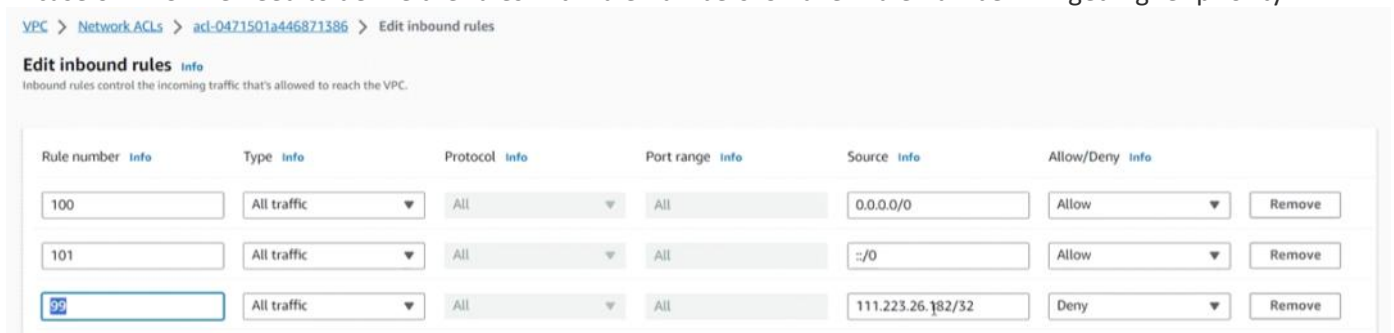


Now private server will be able to access internet.

NACL vs Security Groups -

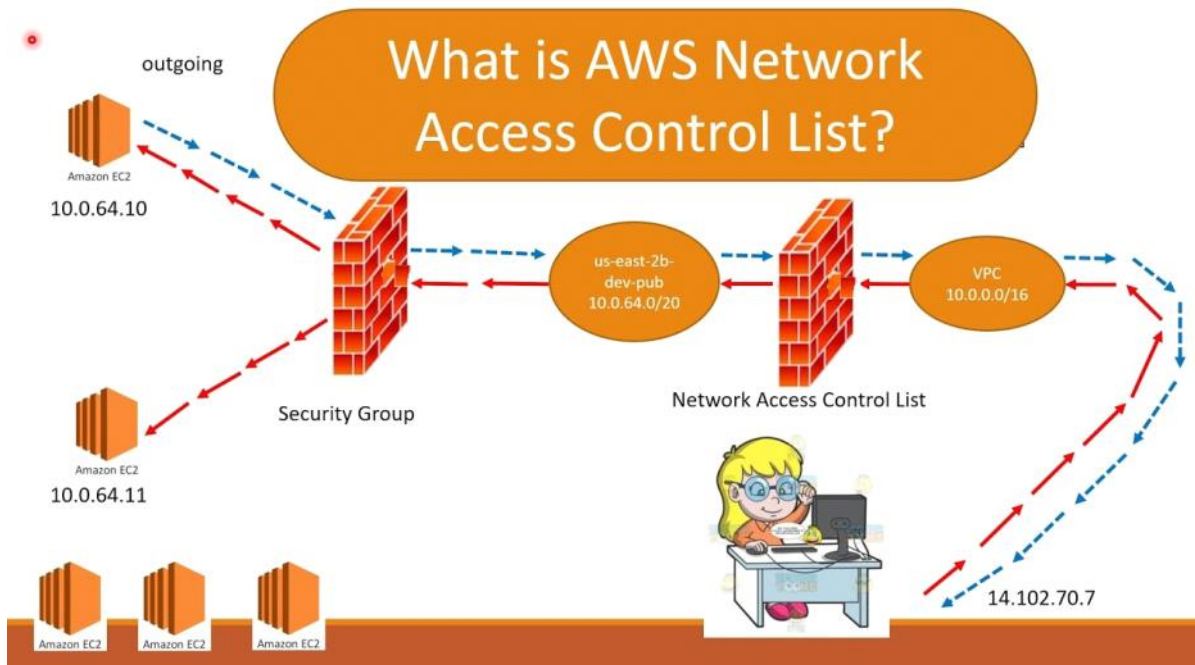
NACL attached to subnet while Security group attached to ec2 instance.

By-default all traffic is denied and using SG we can only allow traffic. Using NACL we can deny specific traffic as well. In case of NACL we need to define the rules with rule numbers. Smaller Rule number will get higher priority.

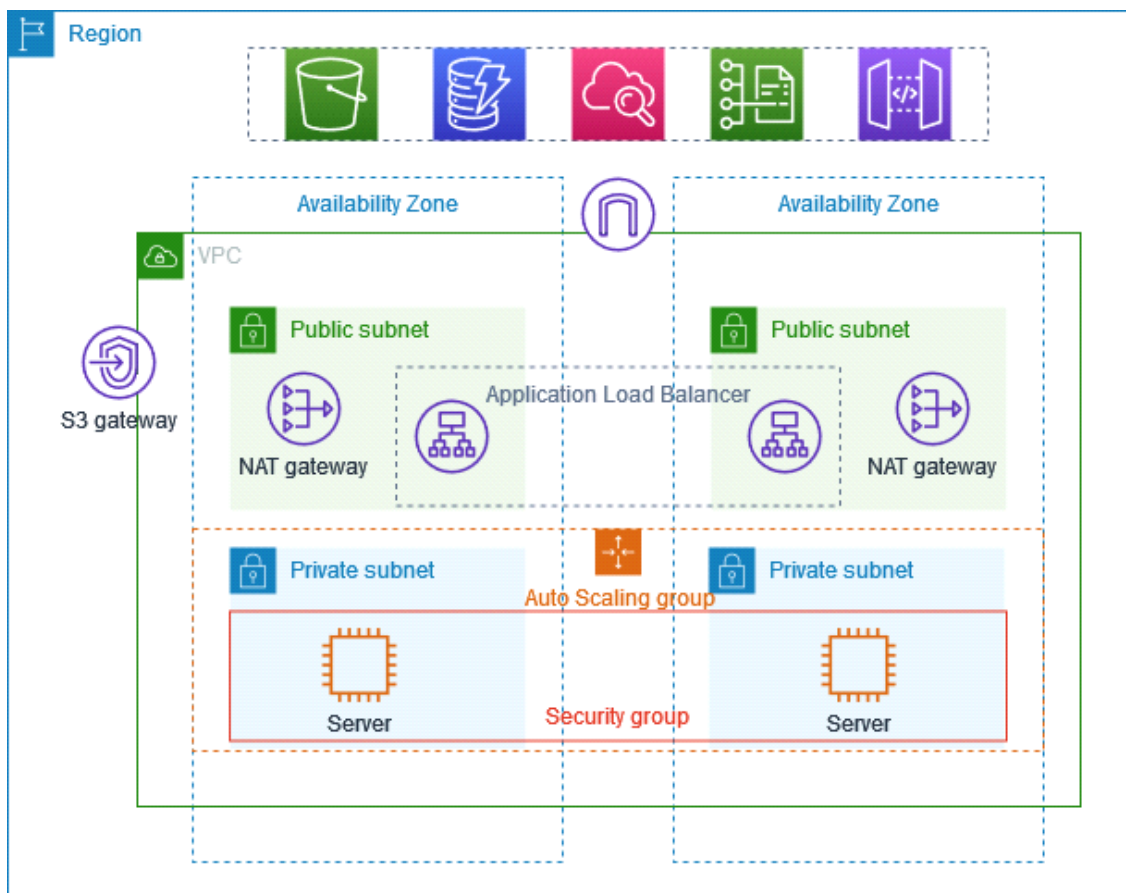


SGs are stateful (if we allow port 80 for inbound then by-default SG will allow outbound from the same port. No need to configure separately.) while NACLs are stateless.

When we create VPC, by-default NACL will also get created which allows all traffic. But when we create our custom NACL then it by-default denies all traffic.



Complete VPC diagram -



VPC Peering -

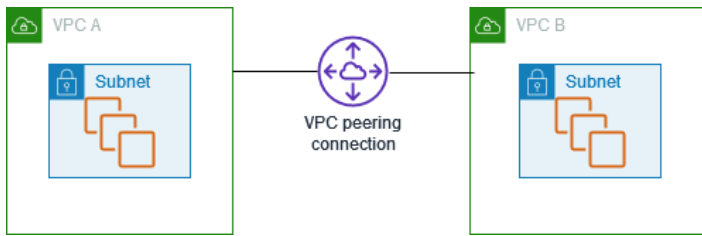
VPC Peering is a networking connection between two Virtual Private Clouds (VPCs) that allows them to communicate with each other privately using private IP addresses, as if they were on the same network.

☒ Simple Definition

VPC Peering = A private bridge between two VPCs.

It lets resources like EC2 instances, databases, containers, etc., in one VPC talk to resources in another VPC without

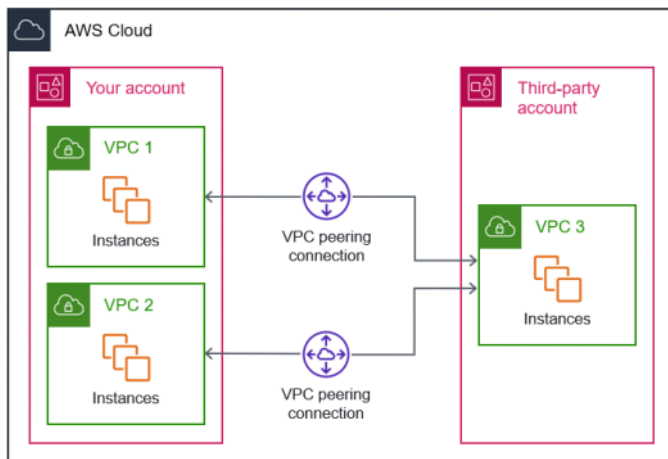
using the public internet.



☒ Use cases

Enable communication between -

- 2 vpcs in same region
- 2 vpcs in diff region
- 2 vpcs in diff account



☒ Key Points

- Works privately using internal IPs.
- Traffic stays inside AWS—no public internet.
- One-to-one connection (between two VPCs).
- Can be between same region or different regions.
- No bandwidth bottleneck—uses AWS backbone.
- No transitive routing (A ↔ B, B ↔ C doesn't mean A ↔ C).
- CIDR ranges must NOT overlap

Steps to create VPC peering connection -

☒ Step 1: Create a VPC Peering Connection

1. In the left menu, click **Peering Connections**
2. Click **Create Peering Connection**
3. Fill in:
 - **Name tag** (optional but recommended)
 - **VPC Requester** (your VPC)
 - **VPC Acceptor**
 - Same account → choose from dropdown
 - Different account → enter Account ID & VPC ID
4. Click **Create Peering Connection**

☒ Step 2: Accept the Peering Request

(Required even if both VPCs are yours)

1. Select the newly created peering connection
2. Click **Actions** → **Accept Request**
3. Status should change to **Active**

☑ Step 3: Update Route Tables

VPC peering won't work until routing is configured.

For each VPC:

1. Go to **Route Tables**
2. Select the route table linked to your Subnet(s)
3. Click **Edit routes → Add route**
4. Add:
 - **Destination** → CIDR of the other VPC
(e.g., if other VPC is 10.2.0.0/16)
 - **Target** → Select the **Peering Connection**
5. **Save changes**

Do the same on both sides.

☑ Step 4: Update Security Groups

Allow traffic between VPCs.

1. Go to **Security Groups**
2. Edit **Inbound rules**
3. Add a rule:
 - Type: e.g., SSH/HTTP/Custom TCP
 - Source: **CIDR of the other VPC** (or specific IPs)
4. Save

(Optional) Update Outbound rules if needed.

☑ Step 5: Test Connectivity

- Ping/SSH/HTTP between EC2 instances in each VPC
- Use **private IPs** only

☑ The Setup

- **VPC1 and VPC2 have overlapping IP ranges**
 - So they **cannot peer with each other**
- **VPC1 ↔ VPC3** ☑ Peering exists
- **VPC2 ↔ VPC3** ☑ Peering exists
- **VPC3 wants to ping an IP that exists in BOTH VPC1 and VPC2**
 - Example: 10.0.1.5 exists in both VPC1 and VPC2

So the BIG question:

🔑 **From VPC3, where will the packet go?**

☑ The Key Rule: Longest Prefix Match

AWS routing — like most IP routing — uses:

☑ Longest Prefix Match

The route with the **most specific subnet mask** wins.

Example:

Route	Prefix Length	More Specific?
10.0.0.0/16	/16	Less specific
10.0.1.0/24	/24	☑ More specific

So /24 wins over /16.

☑ What Happens in This Scenario?

☑ Case 1: VPC3 Route Table Has Different Prefixes

Example:

10.0.1.0/24 → Peering to VPC1
10.0.0.0/16 → Peering to VPC2

Ping to 10.0.1.5:

☑ **Goes to VPC1** (because /24 is longer match than /16)
Even though VPC2 ALSO has that IP, routing ignores it.

☑ Case 2: Both Routes Have the SAME Prefix Length

Example:

10.0.0.0/16 → Peering to VPC1
10.0.0.0/16 → Peering to VPC2

AWS will NOT allow this. ✗

You cannot have two identical destination prefixes pointing to different targets in a route table.

So VPC3 **must choose only one**. The other route can't be added.

Result:

☑ Traffic will go ONLY to the VPC that has the accepted route

✗ The other VPC becomes unreachable

AWS Transit Gateway -

☑ What is AWS Transit Gateway?

AWS **Transit Gateway (TGW)** is a **central networking hub** that allows you to connect multiple:

- VPCs
- On-premises networks (via VPN or Direct Connect)
- AWS accounts
- Regions (using inter-region peering)

Think of it as a **network router in the cloud** that simplifies complex connectivity.

☑ Simple Definition

Transit Gateway = A central router that connects many networks together.

Instead of building **many-to-many** peering connections, everything connects to **one place** — the Transit Gateway.

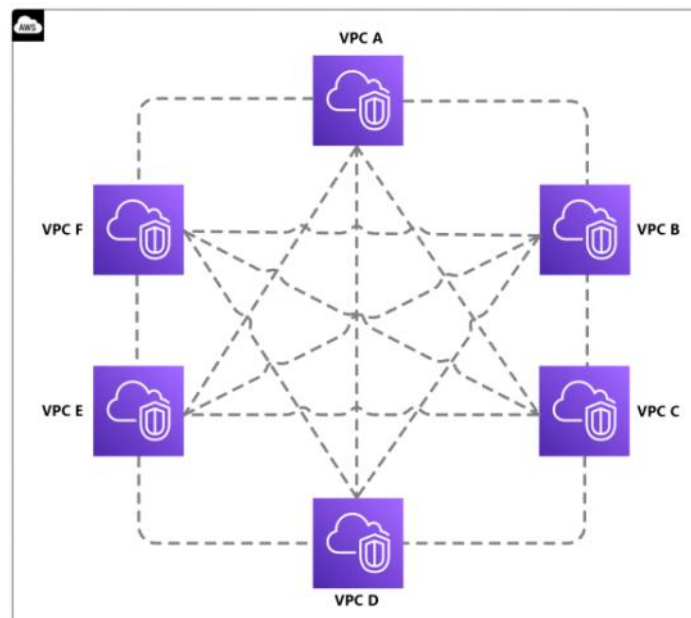
☑ Why Do We Need It?

Without Transit Gateway (Using VPC Peering)

- Full mesh needed
- Connections grow exponentially

For **N VPCs**:

Connections required = $N(N-1)/2$



Example:

- 6 VPCs → 15 peering links ☢
- Routing becomes messy
- No transitive routing

With Transit Gateway

All VPCs connect to **one hub**, so routing becomes simple:

Each VPC → Transit Gateway → Any other VPC

For 6 VPCs → ☑ Only 6 attachments

☑ Supports **transitive routing**

VPC A → TGW → VPC B → TGW → VPC C

☑ Steps to Create a Transit Gateway

☑ Step 1: Open VPC Dashboard

- Login to AWS Console
- Go to **VPC** service
- On the left side, click **Transit Gateways**

☑ Step 2: Create Transit Gateway

1. Click **Create Transit Gateway**
2. Fill details:
 - **Name tag:** (e.g., corp-tgw)
 - **Description:** optional
 - **Amazon ASN:** leave default unless using DX/VPN routing
 - **DNS support:** Enable
 - **Auto-accept shared attachments:** Optional
 - **Default route table association/propagation:** Optional
3. Click **Create Transit Gateway**

☑ TGW is now created

⌚ It may take ~1–2 minutes to become available

☑ Step 3: Create VPC Attachments

To connect a VPC to the TGW:

1. In VPC dashboard, go to **Transit Gateway Attachments**
2. Click **Create Transit Gateway Attachment**
3. Select:
 - **Name tag**
 - **Transit Gateway** (the one you created)
 - **Attachment type** → **VPC**
 - **VPC** (choose the VPC to attach)
 - **Subnets** (select at least 1 subnet per AZ you want routing in)
4. Click **Create Attachment**

Repeat this step for every VPC you want to connect.

☑ Example: Attach VPC-A, VPC-B, VPC-C

☑ Step 4: Configure TGW Route Tables

1. Go to **Transit Gateway Route Tables**
2. Select the default or create a new one
3. Add **routes**:
 - **Destination:** CIDR of VPC-A → **Target:** Attachment A
 - **Destination:** CIDR of VPC-B → **Target:** Attachment B
 - etc.

You can also **associate** and **propagate** each attachment to the route table.

☑ This tells the TGW how to forward traffic between VPCs.

☑ Step 5: Update VPC Route Tables

For each VPC:

1. Go to **Route Tables**
2. Select the route table for the subnets that need connectivity
3. Click **Edit routes** → **Add route**
4. Add:
 - **Destination:** CIDR of the other VPC
 - **Target:** The **Transit Gateway**

VPC > Route tables > rtb-0c4ec03241747e9c9 > Edit routes

Edit routes

Destination	Target	Status
10.1.0.0/16	Q local X	Active
Q 192.168.0.0/16 X	Q tgw- tgw-06b1d17c8109007901 (tgw-attachment-vpc1) X	-

Add route

Example in VPC-A:

10.20.0.0/16 → Target: Transit Gateway

Repeat for VPC-B, VPC-C, etc.

☑ This tells each VPC to send traffic to the TGW.

☑ What Are VPC Flow Logs?

VPC Flow Logs is an AWS feature that **captures information about network traffic going to and from**:

- A **VPC**
- A **Subnet**
- A **Network Interface (ENI)** (e.g., EC2 instance)

It tells you **who talked to whom, on which port, using which protocol, and whether the traffic was allowed or denied**.

Think of it as **network CCTV** for your VPC.

VPC Endpoints -

☑ What Are VPC Endpoints?

A **VPC Endpoint** allows your resources (like EC2 instances) inside a VPC to **privately** access AWS services **without using the public internet, without NAT Gateway, without Internet Gateway (IGW), and without a public IP**.

Think of it as a **private doorway** from your VPC to AWS services.

Example - In hotels we can call any room service by some number. That call never leaves hotel. It stays in hotel only.

☑ Why Do We Need VPC Endpoints?

Without an endpoint, traffic to AWS services like S3 or DynamoDB:

EC2 → NAT Gateway/IGW → Public Internet → AWS Service

With a VPC Endpoint (never leaves AWS) :

EC2 → Private AWS Network → AWS Service

☑ Types of VPC Endpoints

There are **two main types**:

☑ 1 Interface Endpoint

(AKA PrivateLink Endpoint)

- Uses an **Elastic Network Interface (ENI)** in your subnet
- Connects to services over **PrivateLink**
- Traffic goes through a **private IP**
- Supported for MANY AWS services and **third-party services**

☑ 2 Gateway Endpoint

- Works only for TWO services:
 - ☑ **Amazon S3**
 - ☑ **DynamoDB**
- Adds a route entry in your route table
- No ENI created
- No PrivateLink

Cost:

☑ **FREE** (no hourly or data cost)

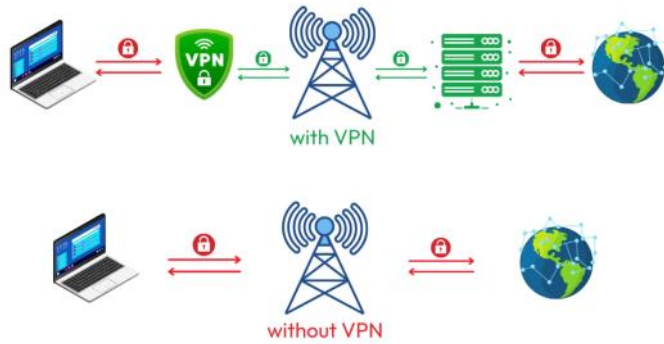
☑ **Do VPC Endpoints Use Public Internet? => NO**

☑ **Do You Need NAT Gateway or IGW? => No**

☑ Common Use Cases

- ☑ Access S3 privately from private subnets
- ☑ Access SSM/SSM Agent without Internet
- ☑ Private communication between VPCs and SaaS providers
- ☑ Reduce NAT Gateway costs
- ☑ Meet security/compliance requirements

VPN -



A VPN (Virtual Private Network) is a secure communication tunnel that allows private traffic to travel safely over an untrusted or public network (like the internet). Let's break it down clearly:

✓ What is a VPN?

A **VPN creates an encrypted tunnel** between:

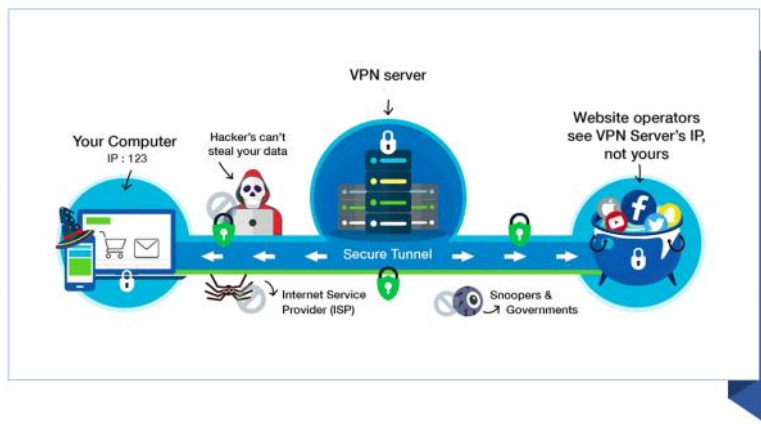
🔗 Your device (client)

and

🔗 Another network (like corporate network, cloud VPC, or VPN server)

This tunnel ensures that:

- ✓ Your data is secure
- ✓ Your identity (IP address) is hidden
- ✓ No one in between can read or modify the traffic



✓ How VPN Works (Step-by-Step)

1 Client Initiates Connection

Your laptop/office router/VPN client sends a request to the VPN server.

2 Authentication

The VPN verifies identity using:

- Username/password
- Certificates
- Keys
- MFA (optional)

3 Tunnel Establishment

A secure tunnel is created using protocols like:

- **IPsec** (commonly used for Site-to-Site VPNs)
- **OpenVPN**
- **WireGuard**
- **SSL/TLS**
- **IKEv1/IKEv2**

4 Encryption

All data passing through the tunnel gets encrypted. Even if someone intercepts it, it's unreadable.

5 Traffic Routing

Once the tunnel is up:

- Your traffic is routed to the VPN server
- The VPN server forwards the traffic to destination (Internet or private network)

To outsiders, it appears that traffic originates from the **VPN server**, not you.

☒ **Types of VPN**

☒ **1. Site-to-Site VPN**

Used between two networks.

Example:

- Corporate Data Center ↔ AWS VPC
- Branch Office ↔ HQ

Used in organizations.

☒ **2. Client-to-Site (Remote Access) VPN**

Used by individuals.

Example:

- Work-from-home employee connecting to office network

S3 (Simple Storage Service)

21 November 2025 13:26

S3 buckets are region scoped but bucket name should be globally unique.

General configuration

AWS Region

Asia Pacific (Mumbai) ap-south-1

Bucket type [Info](#)

- ☒ **General purpose**
Recommended for most use cases and access patterns. General purpose storage classes that redundantly store objects across multiple Availabilit

Bucket name [Info](#)

demo-aws-s3-bucket

✗ Bucket with the same name already exists

Bucket names must be 3 to 63 characters and unique within the global namesp

Copy settings from existing bucket - optional

Only the bucket settings in the following configuration are copied.

[Choose bucket](#)

Format: s3://bucket/prefix

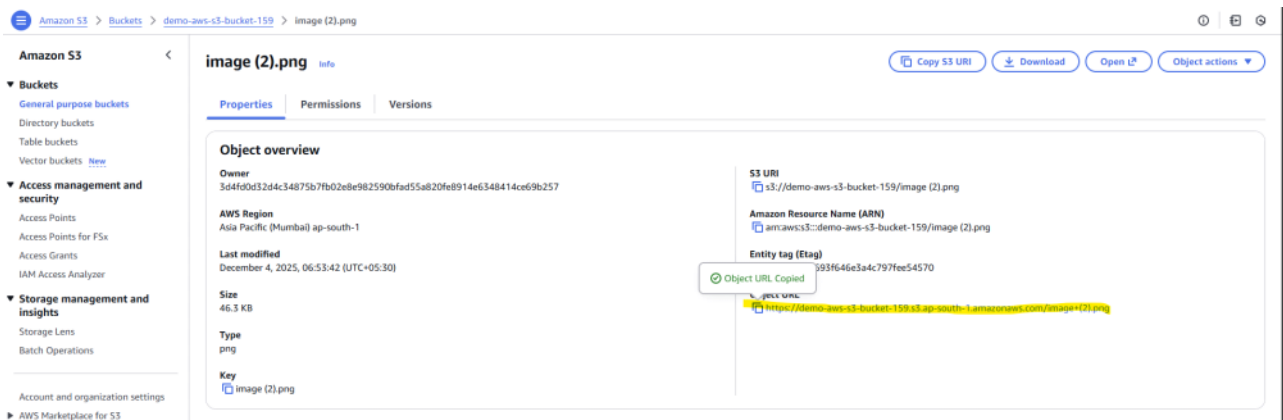
Bucket naming rules -

General purpose buckets naming rules

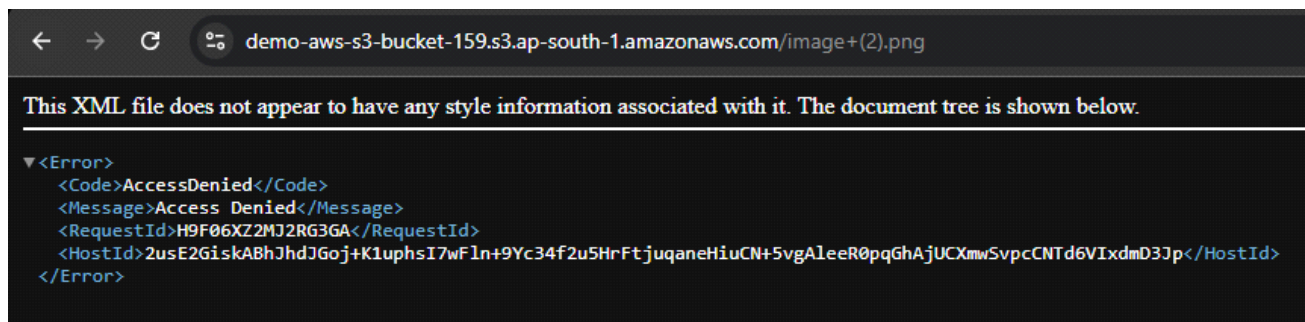
The following naming rules apply for general purpose buckets.

- Bucket names must be between 3 (min) and 63 (max) characters long.
- Bucket names can consist only of lowercase letters, numbers, periods (.), and hyphens (-).
- Bucket names must begin and end with a letter or number.
- Bucket names must not contain two adjacent periods.
- Bucket names must not be formatted as an IP address (for example, 192.168.5.4).
- Bucket names must not start with the prefix xn--.
- Bucket names must not start with the prefix sthree-.
- Bucket names must not start with the prefix amzn-s3-demo-.
- Bucket names must not end with the suffix -s3alias. This suffix is reserved for access point alias names. For more information, see [Access point aliases](#).
- Bucket names must not end with the suffix --o1-s3. This suffix is reserved for Object Lambda Access Point alias names. For more information, see [How to use a bucket-style alias for your S3 bucket Object Lambda Access Point](#).
- Bucket names must not end with the suffix .mrp. This suffix is reserved for Multi-Region Access Point names. For more information, see [Rules for naming Amazon S3 Multi-Region Access Points](#).
- Bucket names must not end with the suffix --x-s3. This suffix is reserved for directory buckets. For more information, see [Directory bucket naming rules](#).
- Bucket names must not end with the suffix --table-s3. This suffix is reserved for S3 Tables buckets. For more information, see [Amazon S3 table bucket, table, and namespace naming rules](#).
- Buckets used with Amazon S3 Transfer Acceleration can't have periods (.) in their names. For more information about Transfer Acceleration, see [Configuring fast, secure file transfers using Amazon S3 Transfer Acceleration](#).

After creating a bucket with basic settings, now we can upload files to the bucket. The content inside bucket is called Object. If we try to access the object url



We will get access denied -



To resolve this error -

1) We need to allow public access -

Uncheck the below checkbox -

Edit Block public access (bucket settings) [Info](#)

Block public access (bucket settings)

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access is blocked, AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your application customizes the individual settings below to suit your specific storage use cases. [Learn more](#)

☒ Block all public access

Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

☒ Block public access to buckets and objects granted through *new* access control lists (ACLs)

S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects.

☒ Block public access to buckets and objects granted through *any* access control lists (ACLs)

S3 will ignore all ACLs that grant public access to buckets and objects.

☒ Block public access to buckets and objects granted through *new* public bucket or access point policies

S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies.

☒ Block public and cross-account access to buckets and objects through *any* public bucket or access point policies

S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

2) Add bucket policy -

Bucket policy

The bucket policy, written in JSON, provides access to the objects stored in the bucket.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Statement1",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::s3-demo-bucket-159/*"
    }
  ]
}
```

Now we can access the object using public url.

AWS uses Authorization = IAM Policy + Bucket Policy

- If either denies → Deny wins (explicit deny)
- If both allow → Access allowed
- If one allows and other is silent → Allow

Bucket policy and IAM policy work together, not replace each other.

AWS S3 Versioning

AWS S3 Versioning is a mechanism that **keeps multiple versions of the same object** inside a bucket. It helps you **recover files from accidental deletes, overwrites, corruption, or malicious changes**.

✓ 1. What Does Versioning Do?

When versioning is enabled:

- Every time you upload a new file **with the same key**, S3 **creates a new version** instead of replacing it.
- If you delete an object, S3 **adds a delete marker** — it does NOT remove existing versions.
- You can restore an old version anytime.

✓ 2. Versioning States

S3 buckets have *three* possible versioning states:

State	Meaning
Unversioned (Default)	No version history. Overwrites destroy previous file.
Enabled	Versioning is turned on. S3 stores all versions.
Suspended	Versioning stops making new versions but retains old ones. New uploads behave like unversioned (overwrite).

✓ 3. How Versioning Works Internally

S3 stores every version as a separate **object** with a unique VersionId.

Example:

mybucket/photo.jpg?versionId=121

mybucket/photo.jpg?versionId=122

mybucket/photo.jpg?versionId=123

Even though the key is the same, S3 maintains a **stack** of versions.

✓ 4. Delete Marker (Very Important for Interviews)

When you run:

```
aws s3 rm s3://mybucket/photo.jpg
```

S3 **does NOT delete** the file.

It adds a **delete marker**, which hides previous versions.

photo.jpg (versionId = delete-marker)
photo.jpg (versionId = v3)
photo.jpg (versionId = v2)
photo.jpg (versionId = v1)

To restore:

- Remove delete marker → older version becomes visible again.

To permanently delete:

aws s3api delete-object --bucket mybucket --key photo.jpg --version-id <versionID>

✓ 5. Benefits of Versioning

1 Accident Protection

- Recover files accidentally deleted.
- Restore overwritten or corrupted files.

2 Ransomware Protection

- Attackers cannot destroy past versions without version-specific delete rights.

3 Auditing & Change Tracking

- Track file history.

4 Required for Cross-Region Replication (CRR)

- Versioning must be **enabled on both source and destination**.

✓ 6. Downsides of Versioning

⚠ Higher Storage Cost

Every version is a full object copy.

If you upload a 100MB file 10 times → you pay for **1GB**.

⚠ List operations return delete markers and all versions.

✓ 7. Versioning + Lifecycle Policies (Very Important)

To control costs, use lifecycle rules:

- **Delete non-current versions after N days**
- **Move older versions to S3 Glacier or Deep Archive**

Example lifecycle:

```
{
  "Rules": [
    {
      "ID": "ExpireOldVersions",
      "Status": "Enabled",
      "NoncurrentVersionExpiration": {
        "NoncurrentDays": 30
      }
    }
  ]
}
```

🌐 AWS S3 Static Website Hosting — Full Explanation

AWS S3 Static Website Hosting allows you to host **static websites** (HTML, CSS, JS, images) directly from an S3 bucket **without servers, EC2, or web servers like Nginx**.

It is designed only for **static content** — no backend processing, no PHP/Node/Python.

🎯 1. Requirements for Hosting a Static Website

To host a website on S3:

1. Create an S3 bucket
2. Bucket name must match the **domain name** (if using a custom domain):
Example → mywebsite.com
3. Enable **static website hosting** on the bucket
4. Upload your static files (index.html, CSS, JS, images)
5. Make objects publicly readable using:
 - Bucket policy
 - OR CloudFront OAC (recommended)
6. Access the site via:
 - S3 Website Endpoint

- or CloudFront distribution

2. How to Enable Static Website Hosting

Inside the S3 bucket:

Properties → Static website hosting

Options:

- Enable hosting
- Choose **hosting type**:
 - Host a static website
- Enter:
 - **Index document**: index.html
 - **Error document**: error.html (optional)

Once enabled, AWS provides a website endpoint like:

<http://<bucket-name>.s3-website-<region>.amazonaws.com/>

What You Cannot Do on S3 Hosting

- ✗ No server-side scripting
 - No PHP
 - No Python
 - No Node.js
 - No server-side databases
 - ✗ Cannot generate dynamic pages
 - ✗ No HTTP headers rewriting (without CloudFront)
- S3 static hosting is purely **static**.

Note - S3 website endpoint doesn't support HTTPS on custom domains

We can write redirection rules as well -

Redirection rules – optional

Redirection rules, written in JSON, automatically redirect webpage requests for specific content

```
1 {
2   {
3     "Condition": {
4       "KeyPrefixEquals": "docs/"
5     },
6     "Redirect": {
7       "ReplaceKeyPrefixWith": "documents/"
8     }
9   }
10 }
```

Amazon S3 Transfer Acceleration

Amazon **S3 Transfer Acceleration (TA)** is a feature that speeds up **uploading and downloading** large files to S3 **over long distances** using AWS's global edge network (CloudFront edge locations).

1. Why Transfer Acceleration Exists

If a user is located **far from the S3 region**, uploading large files directly to the S3 regional endpoint can be slow due to:

- High network latency
- Network congestion
- Distance between client ↔ S3 bucket

Transfer Acceleration reduces this latency and increases throughput.

2. How It Works (Simple Explanation)

S3 Transfer Acceleration uses **AWS CloudFront's globally distributed edge locations** to receive data **close to the user**.

Workflow:

1. User uploads to an S3 Accelerated endpoint
2. Data enters the nearest CloudFront edge location
3. AWS's **optimized private network** transfers it at high speed to the S3 bucket in the target region

This avoids slow public internet routes.



3. How to Enable Transfer Acceleration

Go to:

S3 Bucket → **Properties** → Transfer Acceleration → Enable

After enabling, AWS provides a new URL:

<bucket-name>.s3-accelerate.amazonaws.com

You must use these URL for acceleration to work.



4. Pricing

Transfer Acceleration is **not free**.

Charges apply when using accelerated endpoints:

- Data upload via S3 accelerated endpoint is charged per GB
- Data download also incurs charges

Uploads from **the same region** may be automatically switched to the default endpoint (no acceleration cost).



5. Example Accelerated Upload (AWS CLI)

aws s3 cp myfile.zip s3://mybucket/ --endpoint-url <https://mybucket.s3-accelerate.amazonaws.com>



S3 Replication – SRR & CRR (Deep Explanation)

Amazon S3 provides **Replication** to automatically copy objects from one S3 bucket to another **after they are uploaded**.

Replication is used mainly for:

- Backup
- Compliance
- Data locality (copy data near users)
- Cross-region disaster recovery
- Aggregation of logs
- Multi-account data sharing

There are **two types**:



1. SRR – Same-Region Replication

✓ **SRR = Replication within the same AWS Region**

Example: us-east-1 → us-east-1

Use Cases

- Compliance requirements that need multiple copies in same region
- Dev/Test separation
- Keeping a backup bucket in the same region
- Aggregate logs into a central bucket

Key Points

- Lower cost because same-region
- No cross-region data transfer charges
- Replication is **asynchronous**
- Requires **versioning enabled** in both buckets
- Can choose which prefixes/tags to replicate



2. CRR – Cross-Region Replication

✓ **CRR = Replication between different AWS Regions**

Example: us-east-1 → ap-south-1

Use Cases

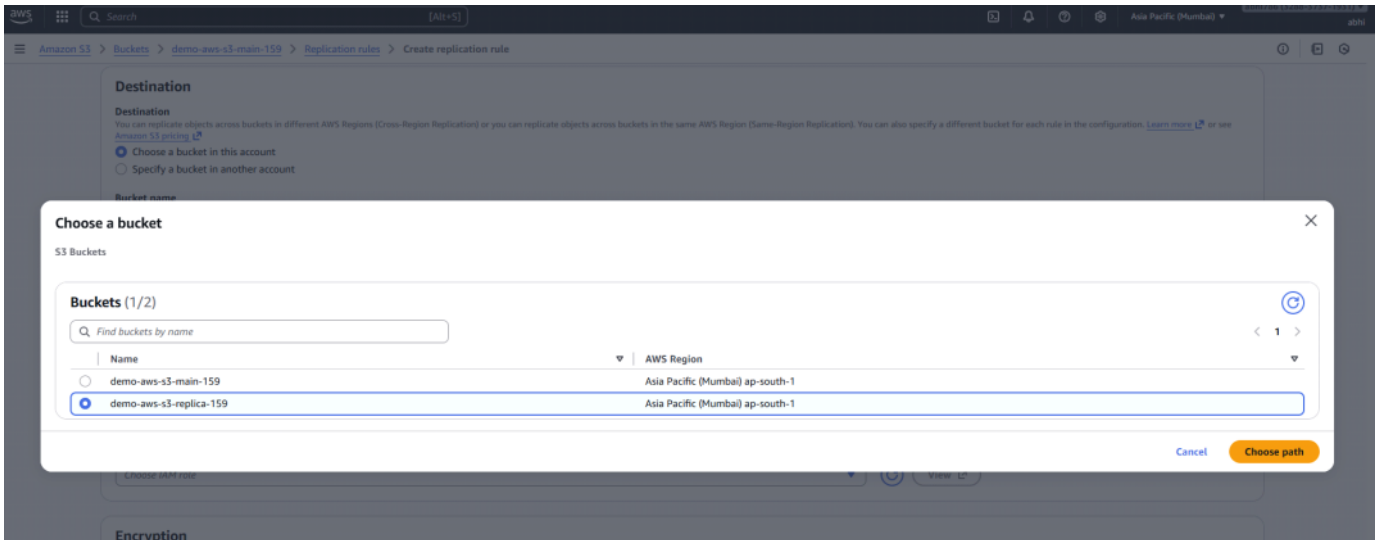
- Cross-region **disaster recovery**
- Reduce latency (copy data close to end users in other regions)
- Compliance (store copy in another region)
- Global content distribution

Key Points

- Higher cost due to:
 - Cross-region transfer charges
 - Storage in destination region
- Asynchronous replication
- Helps meet compliance and DR requirements
- Versioning must be enabled on both buckets

3. How Replication Works (Step-by-step)

1. Source and destination buckets must have versioning enabled
2. You create a **Replication Rule**
3. You assign an **IAM role** for S3 to perform replication
4. When an object is created or updated, S3 automatically copies it to the destination bucket
5. Metadata, ACLs, tags, and object locks may also be copied depending on rule



4. S3 Replication vs Backup

Replication is not backup.

If you delete an object:

- Delete marker is **replicated** (if **delete replication** is enabled)
- Both source and destination lose access unless versioning protects old versions

Backup = independent & isolated copies.

So basically, if we delete the object from main bucket then it will add a delete marker for the same (because versioning is enabled). If we enable **delete replication** then same happens in replicated bucket.

Additional replication options

☐ Replication Time Control (RTC)

Replication Time Control replicates 99.99% of new objects within 15 minutes and includes replication metrics. Additional fees will apply. [Learn more](#)

☐ Replication metrics

With replication metrics, you can monitor the total number and size of objects that are pending replication, and the maximum replication time to the destination Region. You can also view and diagnose replication issues. [Amazon CloudWatch pricing](#)

☒ Delete marker replication

Delete markers created by S3 delete operations will be replicated. Delete markers created by lifecycle rules are not replicated. [Learn more](#)

☐ Replica modification sync

Replicate metadata changes made to replicas from the destination bucket to the source bucket. [Learn more](#)

S3 bucket logging -

Enable logging from Properties --> Server access logging --> enable

Server access logging

Log requests for access to your bucket. [Learn more](#)

Server access logging

☐ Disable
 ☒ Enable

Bucket policy will be updated

When you enable server access logging, the S3 console automatically updates your bucket policy to include access to the S3 log delivery group.

Destination

Specify a destination bucket in the Asia Pacific (Mumbai) ap-south-1 Region. To store your logs under a particular prefix, make sure that you include a slash (/) after the name of the prefix. Otherwise, the prefix will be added to the name of your log files.

Format: s3://<bucket>/<optional-prefix-with-path>

Destination Region

-

Destination bucket name

-

Destination prefix

-

Log object key format

☒ [DestinationPrefix][YYYY-[MM]-[DD]-[hh]-[mm]-[ss]-[UniqueString]]
 ☐ [DestinationPrefix][SourceAccountId]/[SourceRegion]/[SourceBucket]/[YYYY]/[MM]/[DD]/[YYYY]-[MM]-[DD]-[hh]-[mm]-[ss]-[UniqueString]]

To speed up analytics and query applications, use this format.

Log object key example

-

Provide a target bucket (should be in a same region) name where we want to store logs.

Amazon S3 Storage Classes — Comparison Table

Storage Class	Use Case	Availability	Durability	Retrieval Cost	Min Storage Duration	Cost Efficiency Tips
S3 Standard	Hot data, frequently accessed, web apps, content distribution	99.99%	99.999999 11 nines	Low / No extra fee	None	Use for frequently accessed data only; avoid placing cold data here.
S3 Intelligent-Tiering	Unknown/variable access patterns	99.9%	11 nines	Low–Medium (depends on tier)	None	Best when you don't know access patterns; auto-moves to cheaper tiers. Small monitoring fee per object.
S3 Standard-IA (Infrequent Access)	Long-lived but rarely accessed data (DR, backups)	99.9%	11 nines	Medium retrieval fee	30 days	Use for large objects not accessed often. Avoid if you access files frequently.
S3 One Zone-IA	Infrequently accessed data that can tolerate AZ loss	99.5%	11 nines (in a single AZ)	Medium retrieval fee	30 days	20% cheaper than Standard-IA. Use only for non-critical data or replicated data.
S3 Glacier Instant Retrieval	Cold data requiring milliseconds retrieval	99.9%	11 nines	Medium retrieval fee	90 days	Best for archives requiring fast access. Keep only archive-type data here.
S3 Glacier Flexible Retrieval (formerly Glacier)	Very rare access, can wait minutes to hours	99.99%	11 nines	Medium–High retrieval fee	90 days	Ideal for long-term backup; don't use for fast-access workloads.
S3 Glacier Deep Archive	Long-term cold archive (compliance, 7+ years)	99.99%	11 nines	High retrieval cost, hours to retrieve	180 days	Cheapest S3 tier. Use only for data with almost zero access.
S3 Reduced Redundancy Storage (RRS)	Non-critical data	99.99%	99.99% (lower durability)	Low	None	Not recommended anymore; replaced by One Zone-IA.

How to Use S3 Storage Classes Cost-Efficiently

Access Pattern	Recommended Class
Frequently accessed	Standard
Unpredictable access	Intelligent-Tiering
Rare access	Standard-IA
Rare access + non-critical	One Zone-IA
Cold archives + fast retrieval	Glacier Instant Retrieval
Cold archives + slow retrieval	Glacier Flexible Retrieval
Long-term compliance	Glacier Deep Archive

✔ Use Lifecycle Policies

Example lifecycle strategy:

- **Day 0–30** → Standard
- **After 30 days** → Standard-IA
- **After 90 days** → Glacier Flexible Retrieval
- **After 180 days** → Glacier Deep Archive

Saves up to **80%+** on S3 costs.

Note - Storage classes can be applied to only at object level not at bucket level.

✔ 1. Create Lifecycle Rule — AWS Console

Steps

1. Go to **S3** → **Your Bucket** → **Management Tab**
2. Click **Create Lifecycle Rule**
3. Give rule name: Transition-Logs
4. Choose:
 - **Rule scope**: Entire bucket or prefix (e.g., logs/)
5. Add lifecycle actions:
 - **Transition to Standard-IA after 30 days**
 - **Transition to Glacier Deep Archive after 180 days**
6. Add **Expiration** (optional):
 - Delete objects after 365 days
7. Leave other settings default
8. Click **Create Rule**

✔ 2. Lifecycle Rule — JSON Example (Bucket Policy Format)

```
{
  "Rules": [
    {
      "ID": "MoveToIAAndGlacier",
      "Prefix": "logs/",
      "Status": "Enabled",
      "Transitions": [
        {
          "Days": 30,
          "StorageClass": "STANDARD_IA"
        },
        {
          "Days": 180,
          "StorageClass": "DEEP_ARCHIVE"
        }
      ],
      "Expiration": {
        "Days": 365
      },
      "NoncurrentVersionExpiration": {
        "NoncurrentDays": 30
      }
    }
  ]
}
```

✔ 3. AWS CLI Example

```
aws s3api put-bucket-lifecycle-configuration \
--bucket my-example-bucket \
--lifecycle-configuration file:///lifecycle.json
```

Where lifecycle.json contains the JSON above.

☑ 4. Terraform Example

```
resource "aws_s3_bucket" "example" {
  bucket = "my-example-lifecycle-bucket"
}
```

```
resource "aws_s3_bucket_lifecycle_configuration" "example" {
  bucket = aws_s3_bucket.example.id
```

```
  rule {
    id = "transition-to-ia-and-glacier"
    status = "Enabled"
```

```
    filter {
      prefix = "logs/"
    }
```

```
    transition {
      days      = 30
      storage_class = "STANDARD_IA"
    }
```

```
    transition {
      days      = 180
      storage_class = "DEEP_ARCHIVE"
    }
```

```
    expiration {
      days = 365
    }
  }
}
```

▣ What is CORS in AWS S3?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that **controls which domains can access resources** in your S3 bucket using **client-side calls** (JavaScript, React, Angular, etc.).

In short:

CORS defines which external websites are allowed to send requests to your S3 bucket.

It only applies to **browser** → **S3** direct requests.

▣ Why is CORS needed?

Without CORS, browsers **block cross-domain requests** for security.

Example:

- Your website is hosted at: <https://myapp.com>
- Your images are stored in: <https://mybucket.s3.amazonaws.com>

When the browser tries to fetch images, videos, JSON, etc. → **it will be blocked** unless your S3 bucket explicitly allows that origin.

▣ When do you need CORS?

You need CORS when S3 is accessed directly from browser using:

- AJAX / fetch() / Axios
- JavaScript SDK (AWS JS SDK v2/v3)
- React/Angular/Vue apps
- HTML / <video> / <audio>
- Static website frontends using S3/CloudFront

▣ How CORS Works (Simple Example)

If your browser sends a request like:

```
fetch("https://mybucket.s3.amazonaws.com/photo.png")
```

Browser: "This domain is different from myapp.com → **Preflight CORS check**"

Browser sends:

OPTIONS /photo.png
Origin: <https://myapp.com>
S3 responds:

Access-Control-Allow-Origin: <https://myapp.com>
Access-Control-Allow-Methods: GET
If allowed → browser sends actual request.
If not → browser blocks it.

CORS Rules are Configured in the S3 Bucket

CORS is stored as a JSON policy in S3.

Most Common CORS Configurations

1. Allow a Single Domain

```
[
  {
    "AllowedOrigins": ["https://myapp.com"],
    "AllowedMethods": ["GET"],
    "AllowedHeaders": ["*"]
  }
]
```

Use this for production applications.

2. Allow Multiple Domains

```
[
  {
    "AllowedOrigins": [
      "https://app1.com",
      "https://app2.com"
    ],
    "AllowedMethods": ["GET", "POST"],
    "AllowedHeaders": ["*"]
  }
]
```

3. Allow ALL Domains (Not Recommended for Production)

```
[
  {
    "AllowedOrigins": ["*"],
    "AllowedMethods": ["GET", "HEAD"],
    "AllowedHeaders": ["*"]
  }
]
```

Useful for testing.

4. Full CORS with PUT/DELETE (for uploads)

This is needed when your browser uploads files directly to S3.

```
[
  {
    "AllowedOrigins": ["https://myapp.com"],
    "AllowedMethods": ["GET", "PUT", "POST", "DELETE"],
    "AllowedHeaders": ["*"],
    "ExposeHeaders": ["ETag"]
  }
]
```

Where to Configure CORS in AWS S3?

Console:

1. Go to **S3**
2. Open bucket
3. Go to **Permissions**
4. Scroll to **CORS configuration**
5. Paste the JSON

What is a Pre-Signed URL in AWS S3?

A **pre-signed URL** is a **time-limited, permission-limited URL** generated by someone who already has IAM access to an S3 object.

It allows **temporary access** to upload or download an S3 file **without exposing IAM credentials**.

Why is a Pre-Signed URL Needed?

- ◇ **To allow secure temporary access**
 - Give a user access for 5 minutes, 1 hour, or 1 day.
 - After expiry → AWS automatically denies access.
- ◇ **To upload files *directly* from browser / mobile app**
- ◇ **To download files from private S3 buckets**

How a Pre-Signed URL Works (Simple Flow)

Upload Flow

1. User clicks "Upload" on website.
2. Frontend calls backend: *"Give me a pre-signed URL for upload."*
3. Backend uses AWS SDK and generates a 1-hour valid upload URL.
4. Frontend uploads file directly to S3 using that URL.

Download Flow

1. Backend generates a download pre-signed URL.
2. You share it with users.
3. User downloads file from S3 during valid time.

Securing S3 data -

Securing Your AWS S3 Data

At Rest, Server-Side Encryption, and Client-Side Encryption

AWS S3 offers multiple mechanisms to protect your data. These fall into two major categories:

1. Data Encryption At Rest (General Concept)

Encryption at rest means:

- ✓ Data is **encrypted when stored on disk** inside AWS systems
- ✓ Data remains protected even if someone gets access to underlying storage
- ✓ AWS handles decryption only for authorized requests

This is achieved through:

- **Server-Side Encryption (SSE)** — recommended
- **Client-Side Encryption (CSE)** — when you want to encrypt before uploading to AWS

2. Server-Side Encryption (SSE)

AWS encrypts data after receiving it, before writing to disk.

Types of SSE:

SSE-S3 (AES-256) – S3 Managed Keys

AWS **fully manages keys**, rotation, and encryption process.

- ✓ Easy to use
- ✓ No need to manage keys
- ✓ Uses AES-256
- ✓ Enabled by adding a simple header

How to enable:

```
aws s3 cp file.txt s3://mybucket/ --sse AES256
```

Pros:

- Zero key management
- Cheap (free)

Cons:

- Basic security and compliance

SSE-KMS – AWS KMS Managed Keys (Most Common)

Uses **AWS Key Management Service (KMS)** for encryption.

- ✓ Fine-grained key policies

- ✓ Key rotation
- ✓ CloudTrail auditing
- ✓ Ability to use Customer-Managed Keys (CMK)

How to enable:

```
aws s3 cp file.txt s3://mybucket/ --sse aws:kms --sse-kms-key-id <key-id>
```

Pros:

- Most secure SSE option
- Access control: IAM + KMS key policy
- Audit logs for key usage

Cons:

- API calls cost money
- Slightly higher latency due to KMS call

Default encryption [Info](#)

Server-side encryption is automatically applied to new objects stored in this bucket.

Encryption key type [Info](#)

☐ Amazon S3 managed keys (SSE-S3)
☒ AWS Key Management Service key (SSE-KMS)

AWS KMS key [Info](#)

☒ Choose from your AWS KMS keys
☐ Enter AWS KMS key ARN

Available AWS KMS keys

Choose AWS KMS key ▼ [Refresh](#) [Create a KMS key](#)

Bucket Key

When KMS encryption is used to encrypt new objects in this bucket, the bucket key reduces encryption costs by lowering calls to AWS KMS. [Learn more](#)

☐ Disable
☒ Enable

Note - Enable bucket key to reduce api calls directly hitting via kms keys. This will reduce cost by almost 99%.

SSE-C – Customer Provided Keys

You provide your own **encryption key per request**.

AWS:

- Does NOT store your key
- Encrypts using your key
- Deletes key after request

Use case:

- Bring-your-own-key (BYOK) compliance requirements

Cons:

- You must manage key lifecycle
- Hard to scale
- Not available via console

3. Client-Side Encryption (CSE)

You encrypt data before uploading to S3.

AWS stores **already-encrypted data** and cannot decrypt it.

Two methods:

CSE-KMS (Client-Side With KMS)

Your application:

1. Calls KMS to get a Data Encryption Key (DEK)
2. Encrypts the file
3. Uploads encrypted file to S3
4. S3 stores both the encrypted file + encrypted DEK

Pros:

- AWS never sees plaintext
- Strongest protection for extremely sensitive data

Cons:

- Application complexity
- Must implement client encryption libraries

CSE-C (Client-Side Custom Keys)

You manage:

- Key generation
- Key rotation
- Encryption/decryption in your app

AWS stores:

- Only encrypted data
- No knowledge of keys

Pros:

- Full control
- Maximum privacy

Cons:

- Hardest to manage
- No AWS assistance for key recovery

How to copy content from one s3 to another s3 bucket?

☒ 1. Copy Objects Between S3 Buckets Using AWS CLI

A) Copy a Single File

```
aws s3 cp s3://source-bucket/path/file.jpg s3://target-bucket/path/file.jpg
```

B) Copy a Folder (Recursively)

```
aws s3 cp s3://source-bucket/ s3://target-bucket/ --recursive
```

2. Copy Using AWS S3 Console

Good for beginners or one-time copy.

Steps:

1. Open **S3 Console**
2. Go to source bucket
3. Select the files or folders
4. Choose **Actions** → **Copy**
5. Choose **another bucket** → **destination path**
6. Click **Copy**